
A02	Interfaces, Lambdas & Events	Algorithmic Thinking ALD (MTD.ba VZ) SS 2026
Submission via Moodle		24 Points

This assignment covers: interfaces, homogeneous coordinates (Vector3 / Matrix3) implementations, lambda event handlers, and interactive shape selection and transformation.

Submission Requirements:

- Read the `Organizational Guidelines` on Moodle, and adhere to them!
 - Submit a PDF document containing your solution idea, code, and explanations for each exercise
 - Include all source files in a ZIP archive (Maven project)
 - Use the naming convention: `ALD_A02_LASTNAME.pdf` and `ALD_A02_LASTNAME.zip`
-

Exercise 1 (8 points) getCoordinates() for All Shapes

Implement `getCoordinates()` in each shape class listed below. Circle and Line were started in the exercise session — complete them if not done, then implement Ellipse and Star independently.

All implementations must follow this pattern:

1. Build an array of `Vector3` points from the shape's raw coordinates
2. Pass the array through `transformed(points)` — the helper in `Shape` applies the transform matrix to every point if one is set, or returns the array unchanged if not
3. Return `toCoordinates(transformed(points))`

Line (1 point): Two points only — start (`position`) and end (`endX`, `endY`).

Rectangle (1 point): Four corners as `Vector3` points, in order: top-left, top-right, bottom-right, bottom-left.

- Get the origin from `position.getX()` and `position.getY()`
- Top-left: `new Vector3(x, y)`; top-right: `new Vector3(x + width, y)`
- Bottom-right: `new Vector3(x + width, y + height)`; bottom-left: `new Vector3(x, y + height)`

Triangle (1 point): Three vertices as `Vector3` points — one per corner.

- First vertex from `position`: `new Vector3(position.getX(), position.getY())`
- Second and third from `p2` and `p3`: `new Vector3(p2.getX(), p2.getY())` etc.

Circle (2 points): Approximate as a regular polygon with 32 evenly-spaced vertices.

- Center is at `position.getX()`, `position.getY()`
- Each vertex: $p_x = cx + r \cdot \cos(\theta)$, $p_y = cy + r \cdot \sin(\theta)$ where $\theta = \frac{2\pi \cdot i}{32}$

Ellipse (1 point): Same approach as Circle but with separate horizontal and vertical radii:

- $p_x = cx + \text{semiMajorAxis} \cdot \cos(\theta)$
- $p_y = cy + \text{semiMinorAxis} \cdot \sin(\theta)$

Star (2 points): $2 \times \text{numPoints}$ vertices alternating between outer and inner radius:

- Vertex i : $\text{angle} = \frac{\pi}{\text{numPoints}} \cdot i - \frac{\pi}{2}$; $\text{radius} = \text{outerRadius}$ if i is even, innerRadius if i is odd
- $p_x = cx + r \cdot \cos(\text{angle})$, $p_y = cy + r \cdot \sin(\text{angle})$

In all cases, update `draw(GraphicsContext gc)` to call `super.draw(gc)` first, then render with `gc.fillPolygon(coords[0], coords[1], n)` and `gc.strokePolygon(...)`.

Exercise 2 (4 points) Lambda Event Handlers in registerEvents()

Complete the `registerEvents()` method in `ShapeApp`. Register all five mouse event handlers on the canvas using lambda expressions:

- `setOnMouseMoved` → `drawing.handleMouseMoved(x, y)`
- `setOnMousePressed` → `drawing.handleMousePressed(x, y)`
- `setOnMouseDragged` → `drawing.handleDrag(x, y)`
- `setOnMouseReleased` → `drawing.handleMouseReleased(x, y)`
- `setOnMouseClicked` → `drawing.handleClick(x, y)`

In your submitted PDF: show the complete `registerEvents()` method and include a screenshot showing that clicking a shape highlights its bounding box in green and that dragging a `Triangle` moves all three vertices correctly.

Exercise 3 (8 points) Transform Methods in Drawing

Implement the four interactive transform methods in `Drawing.java`. Each method operates on the currently selected shape (`shapes[selectedIndex]`) and must call `redraw()` afterwards. If no shape is selected (`selectedIndex < 0`), the method does nothing.

The toolbar buttons in `ShapeAppBase` already call these methods — once you implement them, the buttons become functional.

rotateSelected(double radians) — 3 points

Rotate the selected shape around its own bounding-box center. Use three composed transforms:

1. Translate to origin: `createTranslation(-cx, -cy)`
2. Rotate: `createRotation(radians)`
3. Translate back: `createTranslation(cx, cy)`

scaleSelected(double fx, double fy) — 2 points

Scale the selected shape around its own bounding-box center. Same three-step pattern as rotation, but use `createScaling(fx, fy)` instead of `createRotation`.

mirrorSelectedHorizontal() — 2 points

Mirror the selected shape horizontally around its own bounding-box center. Same three-step pattern using `createHorizontalMirroring()`.

deleteSelected() — 1 point

Remove the selected shape from the `shapes` array. Shift all shapes after it one position down, decrement count, and reset `selectedIndex = -1`.

TransformFactory Reference:

Method	Effect	Matrix
<code>createTranslation(dx, dy)</code>	shifts point by (dx, dy)	$\begin{pmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{pmatrix}$
<code>createRotation(radians)</code>	rotates counter-clockwise around origin	$\begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$
<code>createScaling(fx, fy)</code>	scales x by fx, y by fy	$\begin{pmatrix} fx & 0 & 0 \\ 0 & fy & 0 \\ 0 & 0 & 1 \end{pmatrix}$
<code>createHorizontalMirroring()</code>	flips over x-axis ($y \rightarrow -y$)	$\begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
<code>createVerticalMirroring()</code>	flips over y-axis ($x \rightarrow -x$)	$\begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Exercise 4 (4 points) Full Interaction Verification

Verify that your implementation works end-to-end with the pre-built toolbar. For each item below, include a screenshot in your PDF that clearly shows the operation working.

Points	Operation	What to show
2 pts	Shape creation	Create all 6 shape types using the toolbar buttons (Circle, Rectangle, Line, Triangle, Ellipse, Star). One screenshot per shape type, or a single screenshot showing all 6 on the canvas.
1 pt	Transforms	Select a shape and apply at least 3 of the 5 toolbar operations: Rotate 45, Scale $\times 1.5$, Mirror H, Scale $\times 0.75$, Delete. Show before and after, or label the operations in your screenshot.
1 pt	Color change	Select a shape, change its fill color via the color picker, then change its stroke color . Include screenshots showing both changes applied.

Note: all toolbar operations are pre-built in `ShapeAppBase`. They only work correctly if your `getCoordinates()` implementations (Exercise 1) and your `registerEvents()` handlers (Exercise 2) are correct.
